

Comprehensive AI & Machine Learning Study Guide

Based on materials from

Ibrahim Sammour
(Info 407 Course)

June 23, 2025

Contents

I	Fundamentals of Machine Learning & Neural Networks	4
1	Core Machine Learning Concepts	4
1.1	Machine Learning Goal	4
1.2	Definitions: A Server Example	4
1.3	Inspiration: The Biological Neuron	4
2	The Artificial Neural Network (ANN)	4
2.1	The Artificial Neuron	4
2.2	Network Structure	5
2.3	Forward and Backward Pass	5
2.4	Generalization and Overfitting	5
II	Building Blocks of a Neural Network	6
3	Activation Functions	6
3.1	Why We Need Non-Linearity	6
3.2	Types of Activation Functions	6
3.2.1	Step Function	6
3.2.2	Linear Function	6
3.2.3	Sigmoid Function	6
3.2.4	Rectified Linear Unit (ReLU)	7
3.2.5	Softmax Function	7
4	Loss Functions	7
4.1	Classification Loss	8
4.1.1	Categorical Cross-Entropy Loss (CCE)	8
4.1.2	Sparse Categorical Cross-Entropy Loss (SCCE)	8
4.1.3	Binary Cross-Entropy Loss (BCE)	8
4.2	Regression Loss	8
4.2.1	Mean Squared Error (MSE)	8
4.2.2	Mean Absolute Error (MAE)	8
4.2.3	Huber Loss	8
III	Training and Optimization	9
5	Evaluation Metrics	9
5.1	Confusion Matrix	9
5.2	Classification Metrics	9
6	Optimization and Backpropagation	10
6.1	Derivatives and Gradients	10
6.2	The Chain Rule and Backpropagation	10
6.3	Gradient Descent and Optimizers	10
6.3.1	Gradient Descent (GD)	10
6.3.2	Stochastic Gradient Descent (SGD)	10
6.3.3	Adam Optimizer	10

7	Regularization: Preventing Overfitting	10
7.1	L1 and L2 Regularization	11
7.2	Dropout	11
7.3	Early Stopping	11
7.4	Data Augmentation	11
7.5	Batch Normalization	11
IV	Advanced Topics	12
8	Genetic Algorithms and Neuroevolution	12
8.1	Introduction to Evolutionary Computation (EC)	12
8.2	Genetic Algorithms (GAs)	12
8.2.1	Core Concepts	12
8.3	Neuroevolution	12
8.3.1	Evolving Network Weights (Fixed Topology)	12
9	Decision Trees	13
9.0.1	Key Questions in Building a Tree	13

Part I

Fundamentals of Machine Learning & Neural Networks

1 Core Machine Learning Concepts

1.1 Machine Learning Goal

The main goal of machine learning is to enable machines to take input data and produce the output of a task (such as classification and regression) by learning patterns from the data.

1.2 Definitions: A Server Example

Imagine having a large server hosting some services. We deploy sensors to measure temperature, humidity, and upload/download rates. The server might function normally or it might fail. Our end goal is to detect and predict failures based on the sensor information.

- **Feature:** Each sensor measurement (e.g., temperature) is called a feature. The combination of all features (temperature, humidity, rates) makes a **feature set**.
- **Sample:** A single complete reading of the feature set at a given time is called a sample (imagine it as a vector).
- **Labels (or Targets, Ground-Truths):** The actual outcomes we are trying to predict, such as "normal" or "failure". These are used to train the ML model.
- **Training:** The process of feeding the model samples (features) and their corresponding labels, and asking the model to update its internal parameters to correctly map inputs to outputs. We can represent this as learning a function $y = f(x)$, where y is the target and x is the feature set.

1.3 Inspiration: The Biological Neuron

Artificial Neural Networks (ANNs) are inspired by the structure of the human brain.

- **Dendrites:** Act like an antenna, receiving signals from other neurons.
- **Cell Body (Soma):** The main control center, which processes the incoming signals.
- **Axon:** Transmits the output signal to other neurons.
- **Synapse:** The connection between neurons. The strength of these connections can change, which is the biological basis for learning. This is called **plasticity**.

2 The Artificial Neural Network (ANN)

2.1 The Artificial Neuron

To mimic biological neurons, we create a mathematical model. An artificial neuron receives multiple inputs, processes them, and produces an output.

- **Weights:** Each input connection has a strength, represented by a numerical value called a weight. This is analogous to the synaptic strength.

- **Bias:** An additional numerical value added to the weighted sum of inputs. It helps to fine-tune the neuron's output and allows the model to fit the data better.

The basic operation of a single neuron is a linear transformation: $y = \mathbf{w} \cdot \mathbf{x} + b$. If we have multiple features, $\mathbf{w}$ and $\mathbf{x}$ are vectors.

2.2 Network Structure

An ANN consists of interconnected layers of neurons.

- **Input Layer:** Receives the initial data (the features of a sample).
- **Hidden Layers:** One or more layers between the input and output layers. They perform most of the computation and feature extraction.
- **Output Layer:** Produces the final result (e.g., a class prediction or a regression value).

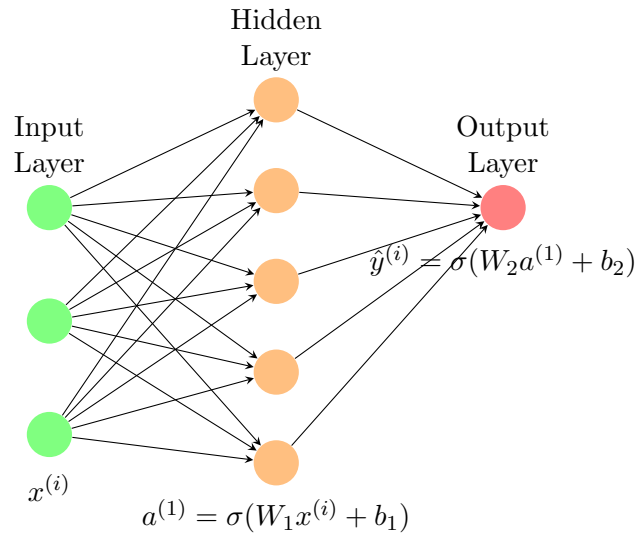


Figure 1: Structure of a simple Artificial Neural Network with one hidden layer.

2.3 Forward and Backward Pass

- **Forward Pass:** The process where input data is fed into the network and travels through the layers, from input to output, to produce a prediction.
- **Backward Pass (Backpropagation):** After a prediction is made, its error (loss) is calculated. This error is then propagated backward through the network to calculate how much each weight and bias contributed to it. These contributions (gradients) are then used to update the parameters to minimize the error.

2.4 Generalization and Overfitting

- **Generalization:** The ability of a model to perform well on new, unseen data that it was not trained on. This is the primary goal of machine learning.
- **Overfitting:** When a model learns the training data too well, including its noise and random fluctuations. It performs exceptionally well on training data but fails to generalize to new data.

Part II

Building Blocks of a Neural Network

3 Activation Functions

3.1 Why We Need Non-Linearity

Activation functions are necessary to introduce non-linearity into a neural network. Without them, a network with multiple layers would behave like a single linear model, no matter how deep it is. A composition of linear functions is just another linear function. Non-linearity allows the network to learn complex patterns and decision boundaries.

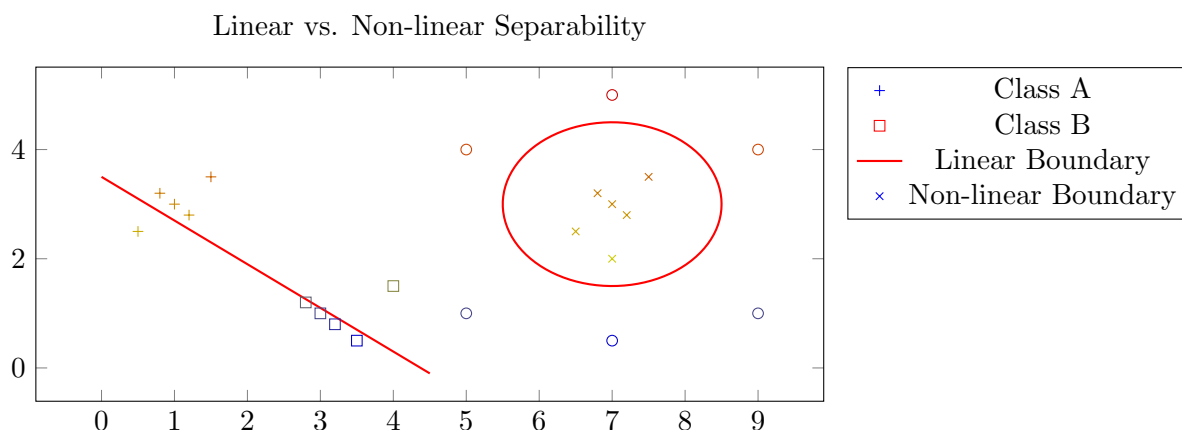


Figure 2: Linear activation functions can only create linear boundaries, while non-linear functions can model complex relationships.

3.2 Types of Activation Functions

3.2.1 Step Function

Mimics a simple on/off neuron. It outputs 1 if the input is positive and 0 otherwise.

$$f(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

Problem: The derivative is zero almost everywhere, making it impossible to use with gradient-based optimization methods like backpropagation.

3.2.2 Linear Function

A simple straight line, $f(x) = x$. **Use Case:** Typically used in the output layer of regression models. **Problem:** If used in hidden layers, the entire network remains linear.

3.2.3 Sigmoid Function

S-shaped function that squishes the output to be between 0 and 1.

$$f(x) = \frac{1}{1 + e^{-x}}$$

Use Case: Commonly used in the output layer for binary classification problems. **Problem:** Suffers from the **vanishing gradient** problem. For very large or very small inputs, the gradient becomes close to zero, slowing down or stopping the learning process.

3.2.4 Rectified Linear Unit (ReLU)

The most popular activation function for hidden layers. It is simple and computationally efficient.

$$f(x) = \max(0, x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

Pros:

- Solves the vanishing gradient problem for positive inputs.
- Computationally fast.

Cons:

- **Dying ReLU problem:** Neurons can get stuck in a state where they only output 0, as the gradient is 0 for all negative inputs. Leaky ReLU, which allows a small, non-zero gradient for negative inputs ($f(x) = \max(0.01x, x)$), is one solution.

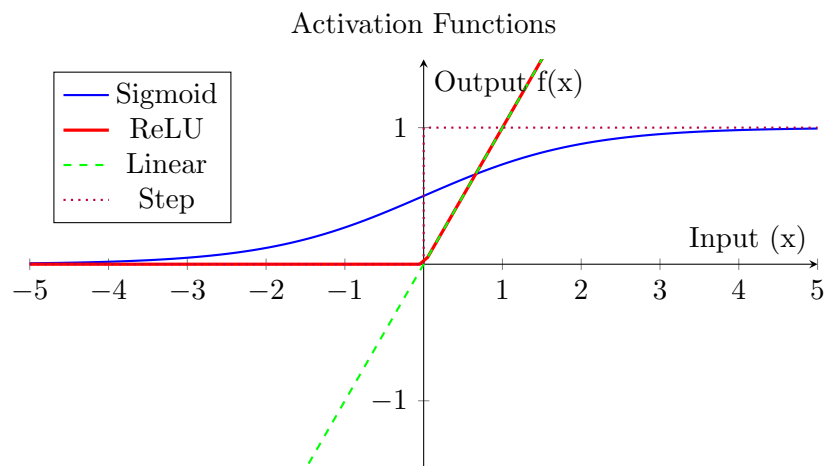


Figure 3: Comparison of common activation functions.

3.2.5 Softmax Function

Used in the output layer for multi-class classification problems. It converts a vector of raw scores (logits) into a probability distribution, where each value is between 0 and 1, and all values sum to 1.

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}$$

Where z_i is the output of the i -th neuron and C is the number of classes.

4 Loss Functions

4.1 Classification Loss

4.1.1 Categorical Cross-Entropy Loss (CCE)

Used for multi-class classification problems where the labels are **one-hot encoded** (e.g., '[0, 1, 0]' for class 2 out of 3). It measures the dissimilarity between the ground-truth probability distribution (y) and the predicted probability distribution ($\hat{y}$).

$$L_i = - \sum_{j=1}^C y_{i,j} \log(\hat{y}_{i,j})$$

Where C is the number of classes. Since only one $y_{i,j}$ is 1 (the true class) and the rest are 0, this simplifies to $L_i = -\log(\hat{y}_{i,\text{true_class}})$. A lower loss means the model assigned a higher probability to the correct class.

4.1.2 Sparse Categorical Cross-Entropy Loss (SCCE)

Functionally the same as CCE, but used when the true labels are integers (e.g., '2' for class 2) instead of one-hot encoded vectors. This is more memory-efficient for problems with a large number of classes.

4.1.3 Binary Cross-Entropy Loss (BCE)

A special case of CCE for binary (two-class) problems.

$$L = -(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}))$$

Where y is the true label (0 or 1) and $\hat{y}$ is the predicted probability for the positive class.

4.2 Regression Loss

4.2.1 Mean Squared Error (MSE)

The most common loss function for regression tasks. It calculates the average of the squared differences between predicted and actual values.

$$L = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

MSE heavily penalizes large errors (outliers) because of the squaring operation.

4.2.2 Mean Absolute Error (MAE)

Calculates the average of the absolute differences between predicted and actual values.

$$L = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

MAE is less sensitive to outliers than MSE.

4.2.3 Huber Loss

A combination of MSE and MAE. It is quadratic (like MSE) for small errors and linear (like MAE) for large errors, providing a balance between sensitivity to outliers and good convergence properties.

Part III

Training and Optimization

5 Evaluation Metrics

Metrics are used to measure the performance of a trained model. Unlike loss functions, they are not used for optimization but for assessing the final model's quality.

5.1 Confusion Matrix

A table used to describe the performance of a classification model.

	Predicted: Positive	Predicted: Negative
Actual: Positive	True Positive (TP)	False Negative (FN)
Actual: Negative	False Positive (FP)	True Negative (TN)

- **TP:** Correctly predicted positive.
- **TN:** Correctly predicted negative.
- **FP (Type I Error):** Incorrectly predicted positive (a "false alarm").
- **FN (Type II Error):** Incorrectly predicted negative (a "miss").

5.2 Classification Metrics

- **Accuracy:** The ratio of correct predictions to total predictions.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

Warning: Accuracy can be misleading for imbalanced datasets.

- **Precision:** Of all the positive predictions, how many were actually correct? High precision is important when the cost of a false positive is high (e.g., spam detection).

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- **Recall (Sensitivity):** Of all the actual positive instances, how many did the model correctly identify? High recall is important when the cost of a false negative is high (e.g., disease diagnosis).

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

- **F1-Score:** The harmonic mean of Precision and Recall. It provides a single score that balances both concerns.

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

6 Optimization and Backpropagation

6.1 Derivatives and Gradients

To minimize the loss function, we need to know how to adjust the weights and biases. The **gradient** tells us the direction of the steepest ascent of the loss function. It's a vector containing the partial derivatives of the loss with respect to each parameter (weight or bias).

$$\nabla L = \left[\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \dots, \frac{\partial L}{\partial b_1}, \dots \right]$$

To decrease the loss, we move the parameters in the **opposite** direction of the gradient.

6.2 The Chain Rule and Backpropagation

Backpropagation is an algorithm that efficiently computes the gradient of the loss function with respect to every parameter in the network. It does this by applying the **chain rule** of calculus, starting from the output layer and moving backward to the input layer.

6.3 Gradient Descent and Optimizers

Optimizers are algorithms that use the calculated gradients to update the network's parameters.

6.3.1 Gradient Descent (GD)

The basic update rule. It computes the gradient using the entire training dataset.

$$w := w - \eta \cdot \nabla L$$

Where η is the **learning rate**, a hyperparameter that controls the step size. **Problem:** Very slow and memory-intensive for large datasets.

6.3.2 Stochastic Gradient Descent (SGD)

Performs a parameter update for each training sample (or a small batch of samples, known as mini-batch GD).

$$w := w - \eta \cdot \nabla L^{(i)}$$

Pros: Much faster and can lead to better generalization by escaping local minima.

6.3.3 Adam Optimizer

A popular, adaptive optimizer that combines the ideas of momentum (which helps accelerate SGD in the relevant direction) and adaptive learning rates (which adjusts the learning rate for each parameter). It is often the default choice for deep learning tasks. The update rule involves maintaining moving averages of both the gradients (momentum) and their squared values.

7 Regularization: Preventing Overfitting

Regularization techniques add penalties to the model to discourage it from becoming too complex and overfitting the training data.

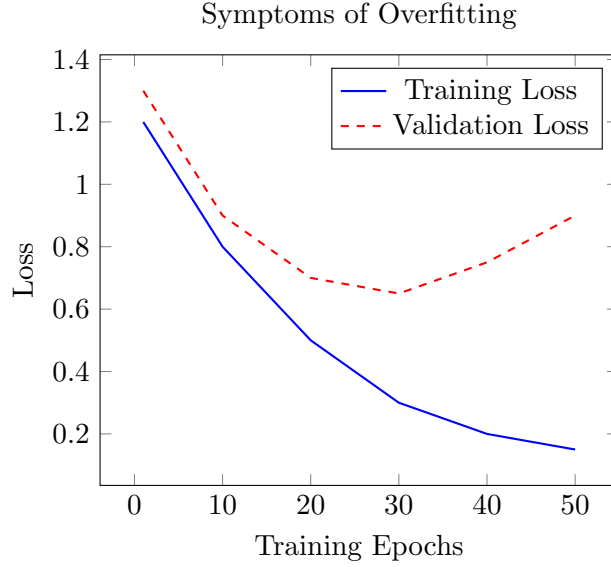


Figure 4: Overfitting occurs when training loss continues to decrease while validation loss begins to increase.

7.1 L1 and L2 Regularization

These methods add a penalty term to the loss function based on the magnitude of the weights.

- **L2 Regularization (Ridge):** Adds the sum of the squared weights. It encourages smaller, more diffuse weight values.

$$L_{\text{reg}} = L + \lambda \sum w_i^2$$

- **L1 Regularization (Lasso):** Adds the sum of the absolute values of the weights. It can lead to sparse models by driving some weights to exactly zero, effectively performing feature selection.

$$L_{\text{reg}} = L + \lambda \sum |w_i|$$

7.2 Dropout

During training, randomly "drops out" (sets to zero) a fraction of neurons in a layer for each forward/backward pass. This prevents neurons from co-adapting too much and forces the network to learn more robust features.

7.3 Early Stopping

Monitor the model's performance on a validation set during training. Stop training when the validation performance stops improving (e.g., the validation loss starts to increase), even if the training loss is still decreasing.

7.4 Data Augmentation

Artificially increase the size and diversity of the training dataset by creating modified copies of existing data (e.g., rotating, cropping, or flipping images).

7.5 Batch Normalization

Normalizes the inputs to a layer for each mini-batch. This helps stabilize and accelerate training by reducing internal covariate shift.

Part IV

Advanced Topics

8 Genetic Algorithms and Neuroevolution

8.1 Introduction to Evolutionary Computation (EC)

EC is a subfield of AI inspired by biological evolution. It uses concepts like selection, crossover, and mutation to iteratively improve a population of candidate solutions to a problem.

8.2 Genetic Algorithms (GAs)

GAs are a foundational type of evolutionary algorithm used for optimization and search problems.

8.2.1 Core Concepts

- **Population:** A collection of candidate solutions ("individuals").
- **Chromosome (Genotype):** The encoded representation of a single solution (e.g., a binary string, a list of numbers).
- **Gene:** A single unit within a chromosome.
- **Fitness Function:** A function that scores how good a solution is. The goal is to maximize fitness.
- **Selection:** The process of choosing "parent" individuals to create the next generation, biased towards those with higher fitness ("survival of the fittest").
- **Crossover (Recombination):** Combining genetic material from two parents to create one or more "offspring".
- **Mutation:** Randomly altering a gene in an offspring with a small probability to introduce new genetic material and maintain diversity.

8.3 Neuroevolution

A specialized field that applies evolutionary algorithms (like GAs) to evolve Artificial Neural Networks. Instead of using gradient-based methods like backpropagation, neuroevolution searches for optimal network parameters (and/or architectures) through a process of evolution.

8.3.1 Evolving Network Weights (Fixed Topology)

The most common approach. The network's architecture (layers, neurons) is fixed. The GA is used to find the optimal set of weights and biases.

- **Chromosome:** A long vector containing all the weights and biases of the network.
- **Fitness:** The performance of the network on a given task (e.g., the score in a game, accuracy on a dataset).

This approach is powerful for tasks where the fitness landscape is noisy, complex, or non-differentiable, such as in reinforcement learning environments (e.g., training an agent to play a game like Flappy Bird).

9 Decision Trees

A brief note on another common machine learning model. A decision tree is a flowchart-like structure where each internal node represents a test on a feature, each branch represents the outcome of the test, and each leaf node represents a class label or a value.

9.0.1 Key Questions in Building a Tree

- **How to choose what feature to split on?** The goal is to choose the feature that best separates the data into "pure" subsets (i.e., subsets containing samples of mostly the same class). This is done by maximizing a purity metric like *Information Gain* or minimizing an impurity metric like the *Gini Index*.
- **When to stop splitting?** A node is not split further if:
 1. It is 100% pure (all samples belong to one class).
 2. Splitting it would result in a tree exceeding a predefined maximum depth.
 3. The improvement in the purity score is below a certain threshold.
 4. The number of samples in the node is below a certain threshold.